Reverse Bits - Line by Line Explanation

ReverseBits Function Explanation (Simplified)
■ What does this function do?
It reverses the bits of a 32-bit integer. That means the leftmost bit becomes rightmost and vice versa.
Example:
If n = 5, its binary is 000...0101
Reversing gives → 1010...0000 (and that's a huge number).
■ Step-by-step breakdown

```
public static long ReverseBits(int n){
long reverseNum = 0;
for(int i = 0; i < 32; i++){
reverseNum <<= 1;
if((n & 1) == 1){
reverseNum++;
}
n >>= 1;
}
return reverseNum;
}
```

Explanation:
1. reverseNum = 0 → holds the reversed bits.
2. Loop 32 times → because int has 32 bits in Java.
3. reverseNum <<= 1 → shifts bits to left to make space.
4. if((n & 1) == 1) → checks if last bit of n is 1.
- If yes, add 1 to reverseNum.
- If no, do nothing.
5. n >>= 1 → moves to next bit by shifting right.
This repeats for all bits, effectively reversing their order.
■ Example (n = 5)
n = 00000000 00000000 00000000 00000101
After reversal = 10100000 00000000 00000000 00000000
Decimal = 2684354560
■ Visualization
Original (n = 5): 00000000 00000000 00000000 00000101
After reversal: 10100000 00000000 00000000 00000000
■ Simple Summary
We start with 0. For each of 32 bits, shift left, copy n's last bit into result, then shift n right.
After 32 loops, bits are completely reversed.
■ Final Result:
ReverseBits(5) = 2684354560